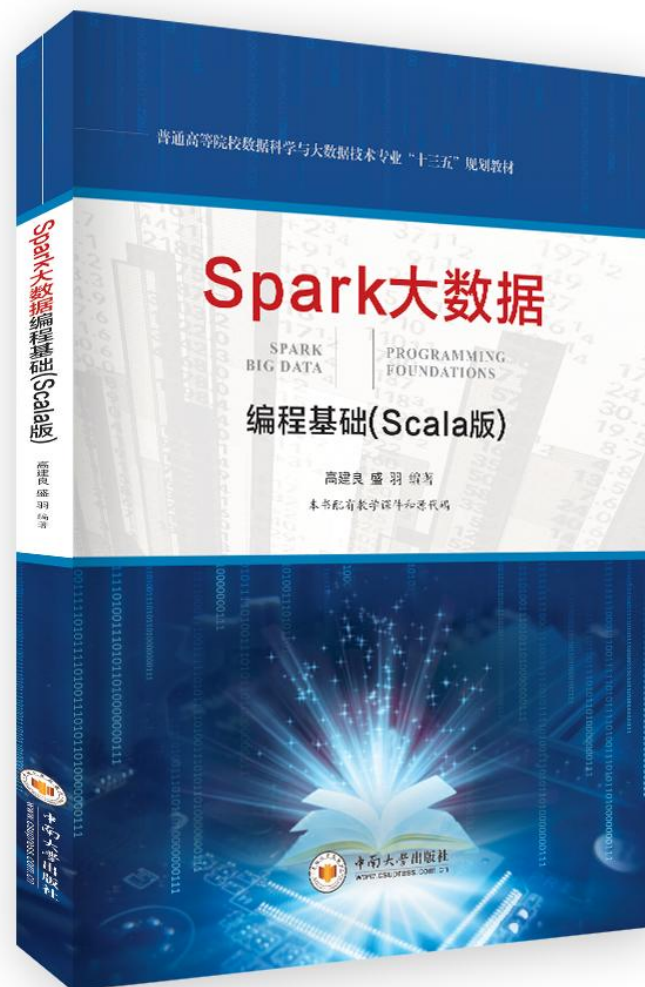




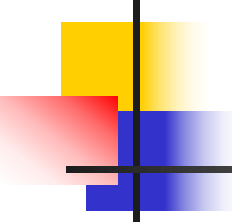
Spark大数据 编程基础 (Scala版)

<http://aibigdata.csu.edu.cn>



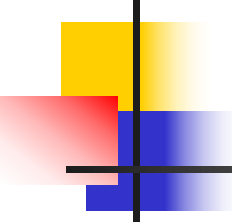
第一章 Spark概述

Spark在大数据处理中发挥着越来越大的作用，本章将对Spark进行概述。首先介绍Spark的发展背景和特点；其次介绍Spark的生态系统，即Spark的核心组件；再对Spark的运行架构进行介绍，为理解Spark的运行流程与原理提供帮助；最后结合代码实例对Spark的编程进行简单介绍。



主要内容

- 1.1 Spark背景
- 1.2 Spark生态系统
- 1.3 Spark运行架构
- 1.4 WordCount示例
- 1.5 本章小结



1.1 Spark背景

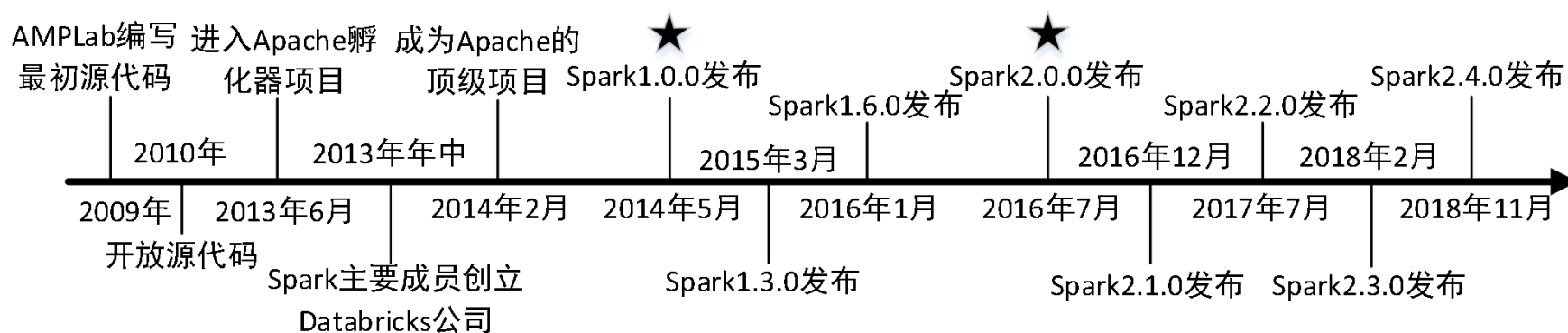
Spark起源于一个学术研究项目，仅用几年时间就已经成为大数据领域应用最广泛的项目之一。**Spark**是一种快速、通用、可扩展的大数据分析引擎。它以高效的方式处理分布式数据集，为分布式数据集的处理提供了一个有效的框架。

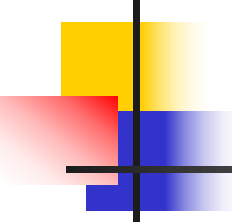
<http://spark.apache.org/>



1.1.1 Spark发展史

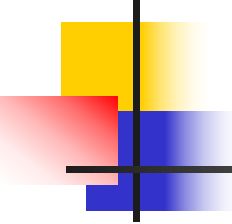
2009年，Spark由Lester和Matei在算法比赛思想碰撞中诞生，此后Spark在加州大学伯克利分校AMP（Algorithms, Machines and People Lab）逐渐形成雏形。Spark的演进路线如图所示。





1.1.2 Spark的特点

Spark的一个含义是“电光火石”，表示运行速度非常快。Spark官网提供的数据表明，如果数据是从内存中读取的话，它的速度可以达到Hadoop MapReduce的100多倍。



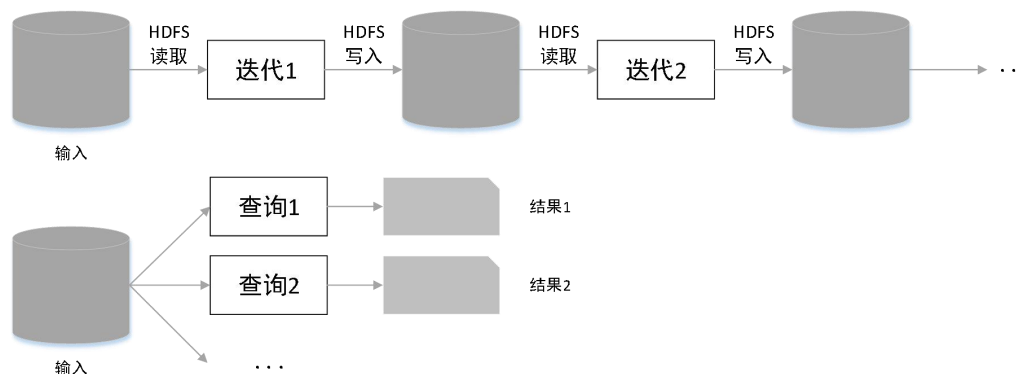
1.1.2 Spark的特点

Spark运行速度快的原因

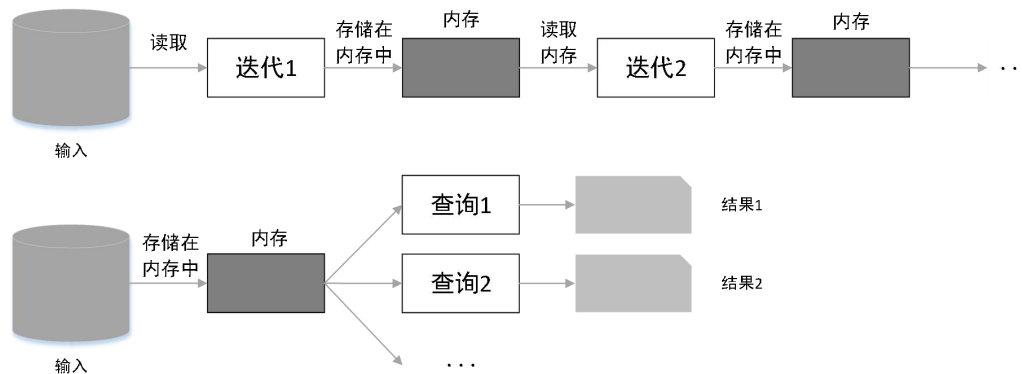
- **Spark**默认情况下的迭代的中间结果放到内存中，后续的运行作业利用这些结果进一步计算。而**Hadoop**的计算结果都需要存储到磁盘中，后续的计算需要从磁盘中读取之前的计算结果。由于从内存中读取数据要比从磁盘读取数据快得多，所以**Spark**运行速度会快很多，尤其需要多次迭代计算情况。

1.1.2 Spark的特点

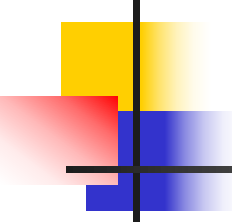
Hadoop与 Spark运行比较



(a) Hadoop MapReduce执行流程



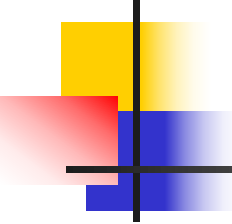
(b) Spark执行流程



1.1.2 Spark的特点

Spark运行速度快的原因

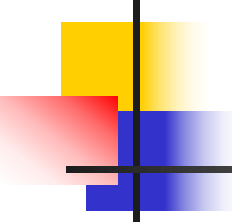
- Spark基于JVM（Java Virtual Machine）进行了优化。Hadoop每次MapReduce操作，启动一个Task便会启动一次JVM，这是基于进程的操作；而Spark每次MapReduce操作是基于线程的，只在启动Executor启动一次JVM，内存的Task操作是在线程复用的。每次启动JVM的时间可能就需要几秒甚至十几秒，那么当Task多了，Hadoop就比Spark花了更多时间。



1.1.2 Spark的特点

Spark的优点

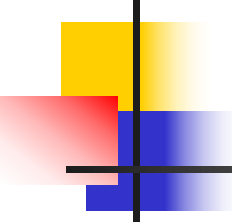
(1) Spark运算效率高。MapReduce在数据Shuffle之前，要花费大量时间排序，而Spark不需要对所有情景进行排序，由于采用DAG（Directed Acyclic Graph，有向无环图）执行计划，每次输出结果可以缓存在内存中，所以迭代运算效率高。对比而言，MapReduce的计算结果保存在磁盘上，上述的比较表明Spark减少迭代过程中数据的落地，提高处理效率。



1.1.2 Spark的特点

Spark的优点

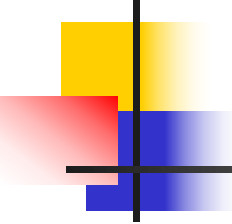
(2) Spark容错性高。Spark引入了弹性分布式数据集（Resilient Distributed Dataset, RDD）概念，它是分布在一组节点中的只读对象的集合。如果某个RDD失败了，可以通过父RDD自动重建，保证容错性。“弹性”是指在任何时候都能进行重算。当集群中的一台机器挂掉而导致存储在其上的RDD丢失后，Spark还可以重新计算出这部分的分区的数。但用户感觉不到这部分的内容丢失过。这样RDD数据集就像块带有弹性的海绵一样，不管怎样挤压都是完整的。



1.1.2 Spark的特点

Spark的优点

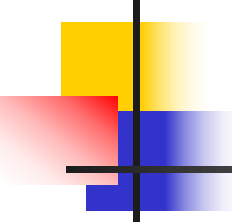
(3) Spark更加通用。 Hadoop只提供Map和Reduce两种操作，而Spark提供的数据集操作的类型很多。转换操作内容有map、filter、flatMap、groupByKey、reduceByKey、union、join、mapValues、sort和partitionBy等操作类型；行动操作包括collect、reduce、save等操作。另外，处理节点间的通信模型也不止有Shuffle一种模式，用户可以命名、物化、控制中间结果的存储、分区。



1.1.2 Spark的特点

Spark的优点

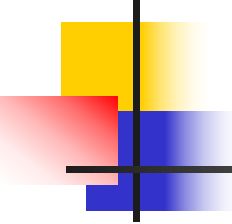
（4）**Spark**提供丰富的**API**，支持多种语言编程，比如Scala、Python、Java以及R语言，便于开发者在熟悉的环境下工作，相同的应用程序**Spark**的代码量比**Hadoop MapReduce**少50%~80%。开发者可以像写单机程序一样开发分布式程序，轻松用**Spark**搭建大数据的平台。



1.1.2 Spark的特点

Spark的优点

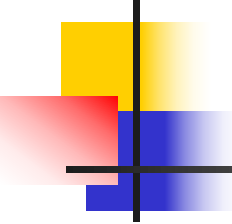
(5) Spark还可以方便地与其他开源产品进行融合。Spark可以使用Hadoop的YARN与Apache Mesos作为它的资源管理和调度器，而且可以处理所有Hadoop支持的数据，包括HDFS、Cassandra、HBase等。这对部署了Hadoop集群的用户来说，不需要数据迁移就可以使用Spark的强大处理能力。Spark也可以不依赖第三方的资源管理和调度器，它可以将Standalone作为其内置资源管理和调度框架，进一步降低了Spark的使用门槛。



1.1.2 Spark的特点

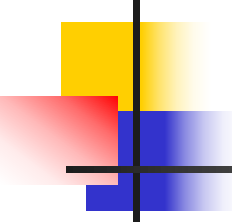
Spark的优点

（6）Spark集批处理、实时流处理、交互式查询与图计算为一体，避免多种运算场景下需要部署不同集群带来的资源浪费。



1.2 Spark生态系统

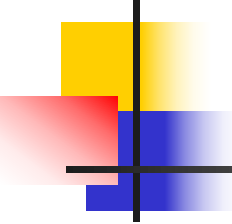
Spark的设计遵循“一个软件栈满足不同应用场景”的理念，逐渐形成了一套完整的生态系统。Spark生态圈以Spark Core为核心，既能够提供内存计算框架，也可以支持SQL即席查询Spark SQL、实时流式计算Spark Streaming、机器学习Spark MLlib和图计算Spark GraphX等。



1.2 Spark生态系统

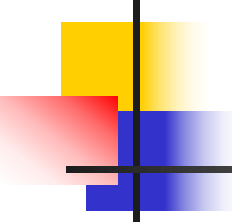
Spark生态系统图

应用层	Spark SQL	Spark Streaming	GraphX	MLlib/ML	SparkR
数据处理引擎	Spark Core				
资源管理层	本地运行模式	Standalone	EC2	Mesos	Hadoop YARN
数据存储层	HDFS,AmazonS3,HBase...				



1.2.1 Spark Core

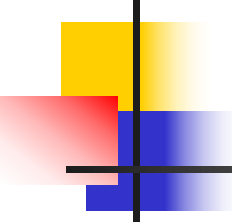
Spark Core是Spark生态系统的核心组件，是一种大数据分布式处理框架。它实现了MapReduce的算子map函数和reduce函数及计算模型，还提供filter、join、groupByKey等丰富的算子，同时，也实现了Spark的基本功能，包括任务调度、内存管理、错误恢复与存储系统交互等模块。



1.2.1 Spark Core

Spark Core 主要特征

(1) Spark Core提供多种运行模式，不仅可以自身运行模式处理任务，如本地模式、**Standalone**模式，还可以使用第三方资源调度框架来处理任务，比如**YARN**、**MESOS**等，通过使用这些部署模式分配资源，可以提高任务的并发执行效率。

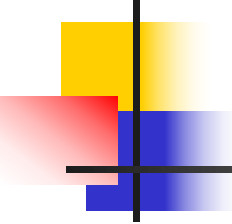


1.2.1 Spark Core

Spark Core 主要特征

（2）Spark Core提供了DAG分布式并行计算框架，而且提供内存机制支持多次迭代计算或数据共享，减少迭代计算时读数据开销，提高迭代计算的性能。

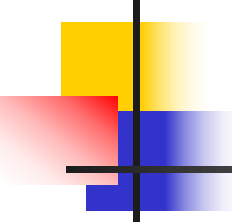
（3）Spark引入弹性分布式数据集（Resilient Distributed Datasets, RDD），实现了应用任务的调度、RPC、序列化和压缩，并为运行在其上层的组件提供API，另外这些对象集合是弹性的，若有部分数据集丢失，它可根据“血统”对丢失部分进行重建，提高数据的容错性。



1.2.1 Spark Core

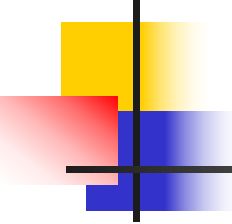
Spark Core 主要特征

（4）Spark优先考虑使用各节点的内存作为存储，当内存不足时才会考虑使用磁盘，这极大地减少了访问磁盘次数，提升了任务执行的效率，使得Spark适用于实时计算、流式计算等场景。



1.2.2 Spark SQL

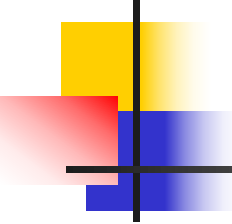
Spark SQL是Spark的一个结构化数据处理模块，可以看作是一个分布式SQL查询引擎。 Spark SQL提供的最核心的编程抽象就是DataFrame。 DataFrame是以指定列（**named columns**）分布式组织数据集，它在概念上等同于关系数据库的一个表或R/Python的一个数据框架，但是提供更精细的优化方式。



1.2.2 Spark SQL

Spark SQL 主要特征

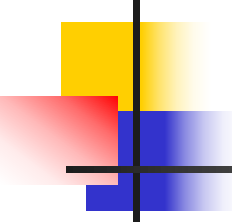
- （1）支持多种数据源：Hive、RDD、Parquet、JSON、JDBC等。
- （2）多种性能优化技术：内存列存储（in-memory columnar storage）、字节码生成技术（byte-code generation）、cost model动态评估等。内存列存储意味着Spark SQL的数据不是使用Java对象的方式来进行存储，而是使用面向列的内存存储的方式来进行存储。也就是说，每一列作为一个数据存储的单位。从而大大优化了内存使用的效率。



1.2.2 Spark SQL

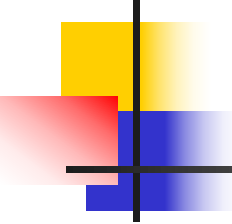
Spark SQL 主要特征

(3) 组件扩展性：对于SQL的语法解析器、分析器以及优化器，用户都可以重新开发，并且动态扩展。



1.2.3 Spark Streaming

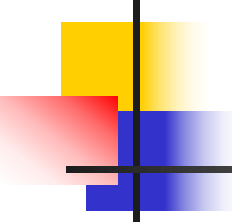
Spark Streaming 是Spark核心API的一个扩展，可以实现高吞吐量的、具备容错机制的实时流数据的处理。支持从多种数据源获取数据，包括Kafka、Flume、Twitter、Kinesis 以及TCP sockets，从数据源获取数据之后，可以使用诸如map、reduce、join和window等高级函数进行复杂算法的处理。最后还可以将处理结果存储到文件系统，数据库和现场仪表盘。Spark Streaming最大的优势是其处理引擎和RDD编程模型可以同时进行批处理和流处理。



1.2.3 Spark Streaming

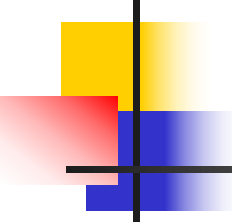
Spark Streaming 工作流程

Spark Streaming将流式计算分解成多个Spark Job，对于每一段数据的处理都会经过Spark DAG图分解以及Spark的任务集的调度过程。对于目前版本的Spark Streaming而言，其最小的Batch Size的选取在0.5~2秒钟之间（Storm目前最小的延迟是100ms左右），所以Spark Streaming能够满足除对实时性要求非常高（如高频实时交易）之外的所有流式准实时计算场景。



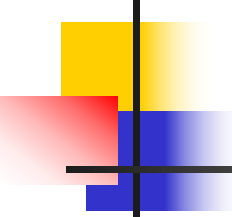
1.2.4 GraphX

GraphX是Spark中图计算的核心组件，便于高效完成图计算的完整的流水作业。在高层次上，GraphX 通过引入一种具有附加到每个顶点和边的属性来扩展 Spark RDD。为了支持图计算，GraphX 公开了一组基本运算符（例如：subgraph, joinVertices和 aggregateMessages）以及Pregel API的优化变体。此外，GraphX还在不断增加其图算法。



1.2.5 MLBase/MLlib

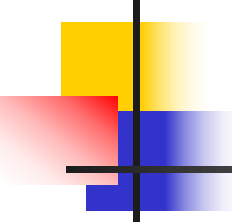
MLBase是Spark生态圈里的一部分，专门负责机器学习，它的目标是降低机器学习的门槛。在性能方面，Spark在机器学习方面提供了高质量的算法，使MLlib能够快速运行，比MapReduce快100倍。同时，MLlib还在发展完善中，不断的有新的机器学习算法加入其中，为大数据编程提供更大的便利。



1.2.5 MLBase/MLlib

MLBase由四部分组成

- （1）MLRuntime：是Spark Core提供的分布式内存计算框架，运行由Optimizer优化过的算法进行数据计算，并且输出结果。
- （2）MLlib：是Spark实现一些机器学习算法和实用程序，比如说分类、回归、聚类、降维、协同过滤还有底层的优化方面。

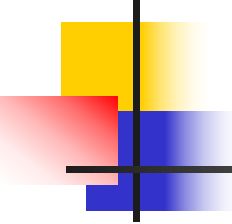


1.2.5 MLBase/MLlib

MLBase由四部分组成

(3) **ML Optimizer**: 它会把用户的数据用它认为最适合的内部已经实现好的机器学习算法以及相关参数处理，并且返回模型或者其他帮助分析的结果。

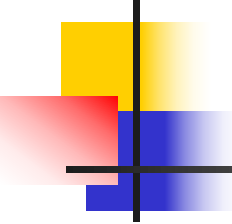
(4) **MLI**: 它是一个特征抽取和高级ML编程抽象算法实现的API的平台。



1.2.6 SparkR

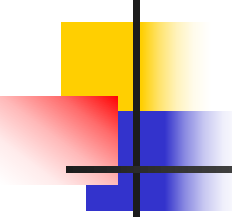
SparkR是AMPLab发布的一个R开发包，为Apache Spark提供了轻量级的前端。SparkR提供了Spark中弹性分布式数据集（RDD）的API，用户可以在集群通过R shell交互性地运行Job。

Spark具有快速、可扩展和交互的特点，R具有统计、绘图的优势，R和Spark的有效结合，解决了R语言中无法级联扩展的难题，同时也丰富了Spark在机器学习方面能使用的Lib库。



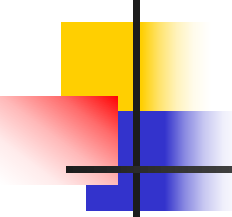
1.3 Spark运行架构

本节对Spark的运行原理、Spark的架构、运行步骤以及运行模式进行介绍，由此可以了解Spark的工作流程。



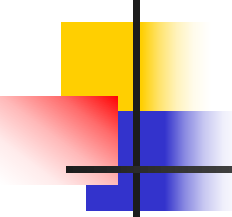
1.3.1 相关术语

- **Client**: 客户端进程，负责提交作业到Master上。
- **Driver**（驱动器节点）：Spark中的Driver运行Application的main函数时生成了SparkContext即Spark的入口，SparkContext的创建是为了准备Spark应用程序的运行环境。
- **Cluster Manager**（集群管理器）：一种外部服务，通过这种外部服务可以在集群上的机器启动应用，用来管理集群。
- **Master**（主节点）：负责接收Client提交的作业，管理Worker，并命令Worker启动Executor。



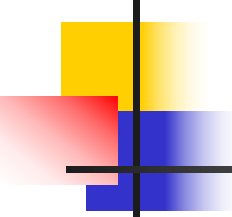
1.3.1 相关术语

- **Application**（应用程序）：是指用户编写的Spark应用程序，它包含**Driver**（驱动器节点）和分布在集群中的多个节点上运行的**Executor**（执行器）代码。
- **Job**（作业）：在Spark运行时，RDD中由**Action**（行动操作）所生成的一个或者多个 **Stage**（调度阶段）。
- **Stage**（调度阶段）：每个Job会根据RDD之间的依赖关系拆分成多组任务集合，称为调度阶段，也叫**TaskSet**（任务集）。Stage的划分是由**DAGScheduler**来进行的。



1.3.1 相关术语

- **Task（任务）**：Stage可以划分成许多个TaskSet，TaskSet包含多个Task。Task被分发到Executor上执行，它是Spark实际执行应用的最小单元。
- **DAGScheduler**：DAGScheduler是面向调度阶段的任务调度器，负责接收Spark应用提交的作业，根据RDD的依赖关系划分Stage，然后提交这些Stage给TaskScheduler处理。
- **TaskScheduler**：TaskScheduler是面向任务的调度器，它接收DAGScheduler提交过来的Stage，然后把Stage拆分成Task，并把Task分发到Worker节点，由Worker节点的Executor来执行该任务。

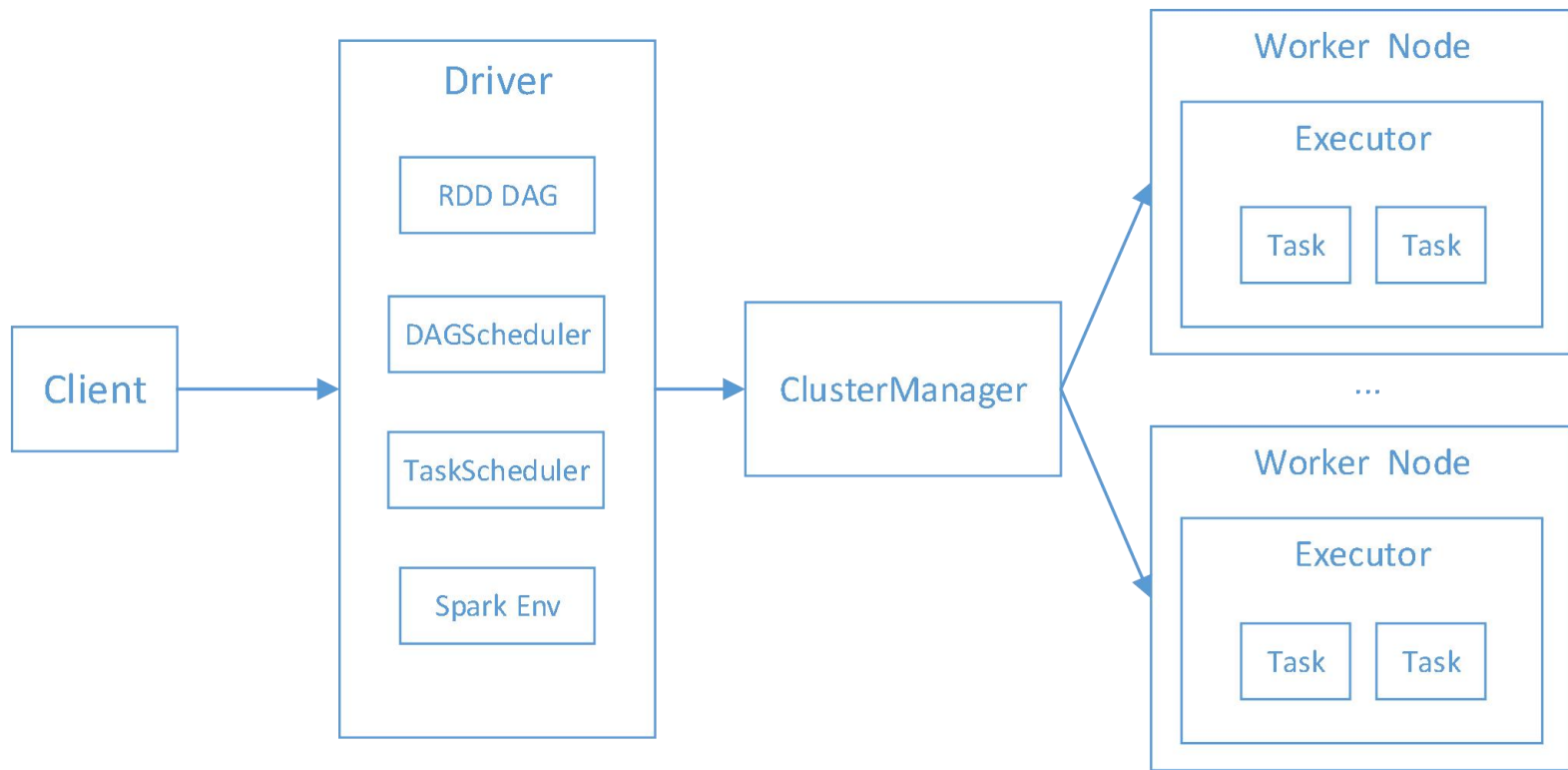


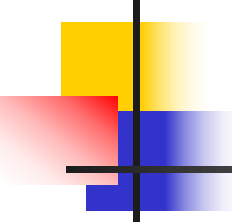
1.3.1 相关术语

- **Block Manager:** 管理RDD的物理分区，每个Block就是节点上对应的物理块，而RDD中的Partition是一个逻辑数据块，对应相应的物理块Block。
- **Task threads:** 任务执行线程。
- **Spark Env:** Spark的执行环境对象，其中包括与众多Executor执行相关的对象。
- **Executor（执行器）:** Application运行在Worker节点上的一个进程，分配到Worker上的任务就是在这里被执行的。

1.3.2 Spark架构

Spark基本架构图

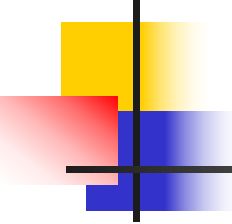




1.3.2 Spark架构

Spark的基本架构

Spark架构采用分布式计算中的**Master-Slave**模型。集群中的主控节点称为**Master**，同样，集群中含有**Worker**进程的节点为**Slave**。**Master**负责控制整个集群的运行；**Worker**节点相当于分布式系统中的计算节点，它接收**Master**节点指令并返回计算进程到**Master**；**Executor**负责任务的执行；**Client**是用户提交应用的客户端；**Driver**负责协调提交后的分布式应用。



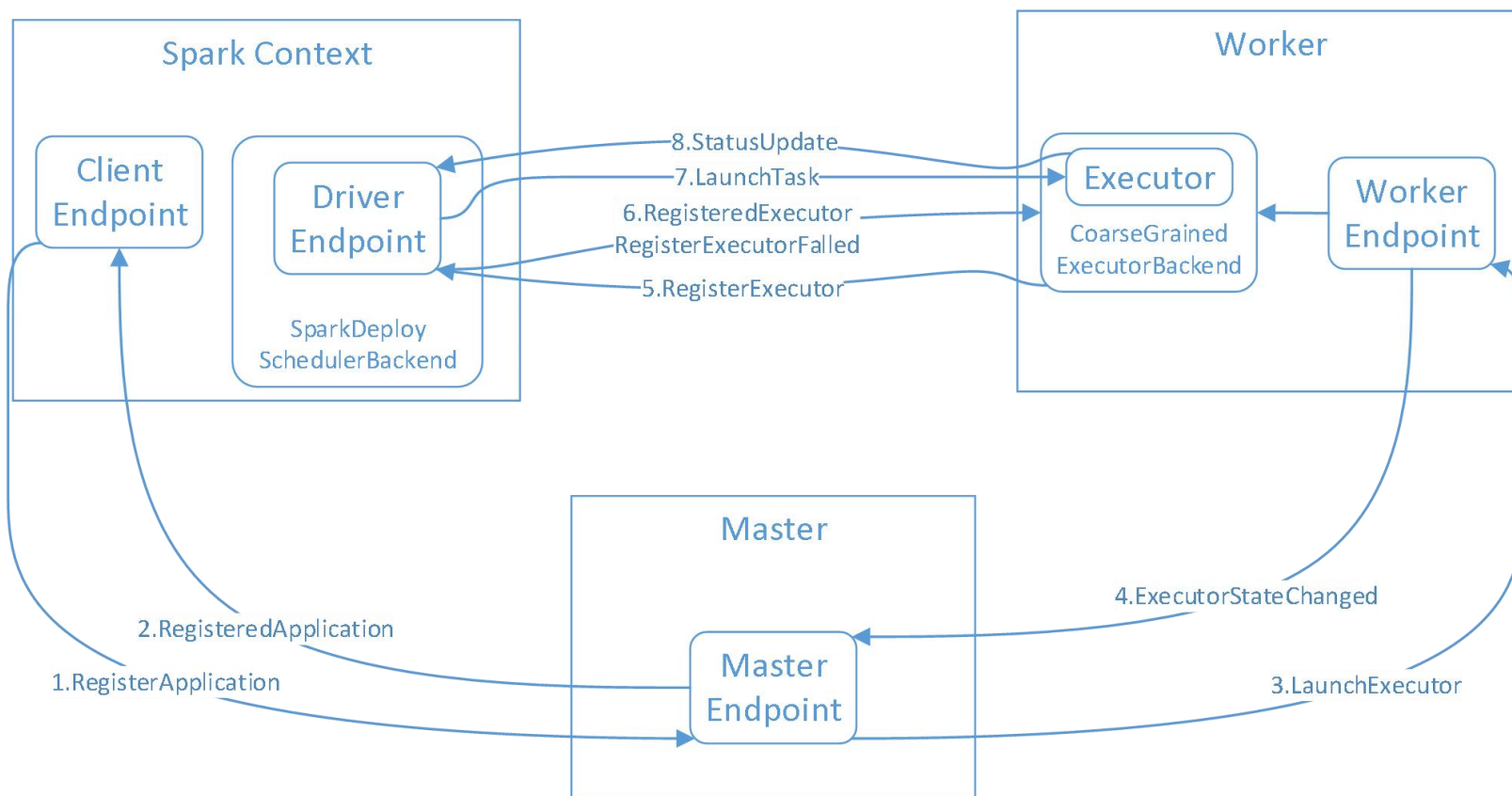
1.3.2 Spark架构

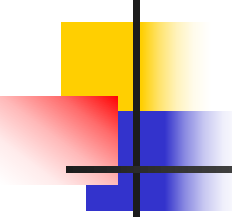
Spark的基本架构

Driver是应用逻辑的起点，负责**Task**任务的分发和调度；
Worker负责管理计算节点，并创建**Executor**来并行处理**Task**任务。**Task**执行过程中所需的文件和包由**Driver**序列化后传输给对应的**Worker**节点，**Executor**对相应分区的任务进行处理。

1.3.2 Spark架构

Spark运行消息通信的交互过程

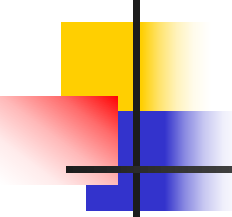




1.3.2 Spark架构

Spark运行消息通信的交互过程

(1) 执行应用程序需启动**SparkContext**，在SparkContext启动过程中会先实例化SchedulerBackend对象，在Standalone（独立运行）模式中实际创建的是SparkDeploySchedulerBackend对象，在该对象的启动中会继承父类DriverEndPoint和创建AppClient的ClientEndpoint的两个终端点。在ClientEndpoint创建的线程池中启动注册线程并向Master发送RegisterApplication注册应用消息。当Master接收到注册应用的消息时，在registerApplication方法中记录应用信息并把该应用加入到等待运行的应用列表中，注册完成后发送成功消息RegisteredApplication给ClientEndpoint，同时调用startExecutorOnWorkers方法运行应用。在执行前需要获取运行应用的Worker，然后发送LaunchExecutor消息给Worker，通知Worker启动Executor。

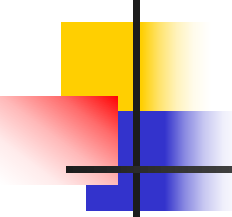


1.3.2 Spark架构

Spark运行消息通信的交互过程

(2) `AppClient.ClientEndpoint`接收到`Master`发送的`RegisteredApplication`消息，需要把注册标识`registered`的值置为`true`，`Master`注册线程获取状态变化后，完成注册`Application`进程。

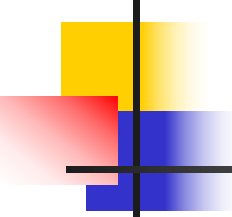
(3) 在`Master`类的`startExecutorOnWorkers`方法中分配资源运行应用程序时，调用`allocateWorkerResourceToExecutors`方法实现在`Worker`中启动`Executor`。当`Worker`收到`Master`发送的`LaunchExecutor`消息，先实例化`ExecutorRunner`对象，在`ExecutorRunner`启动中会创建进程生成器`ProcessBuilder`，然后由该生成器使用`command`创建`CoarseGrainedExecutorBackend`对象，该对象是`Executor`运行的容器，最后`Worker`发送`ExecutorStateChanged`消息给`Master`，通知`Executor`容器已经创建完毕。



1.3.2 Spark架构

Spark运行消息通信的交互过程

- (4) 在Master接收到Worker发送的ExecutorStateChanged消息。
- (5) 在(3)中CoarseGrainedExecutorBackend启动方法onStart发送注册Executor消息RegisterExecutor给DriverEndpoint, DriverEndpoint终端点先判断该Executor是否已经注册, 如果已经存在则发送注册失败消息RegisterExecutorFailed, 否则DriverEndpoint终端点记录该Executor信息, 发送注册成功RegisteredExecutor消息, 在makeOffers()方法中分配运行任务资源, 最后发送LaunchTask消息执行任务。
- (6) 当CoarseGrainedExecutorBackend接收到Executor注册成功RegisteredExecutor消息时, CoarseGrainedExecutorBackend容器中实例化Executor对象。启动完毕后, 会定时向Driver发送心跳消息, 等待接收从DriverEndpoint终端点发送执行任务的消息。

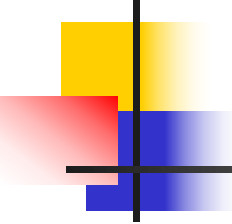


1.3.2 Spark架构

Spark运行消息通信的交互过程

（7）CoarseGrainedExecutorBackend的Executor启动后，接收从DriverEndpoint终端点发送LaunchTask执行任务消息，任务执行是在Executor的launchTask方法实现的。在执行任务时创建TaskRunner进程，由该进程进行任务处理，处理完后发送StatusUpdate消息返回给CoarseGrainedExecutorBackend。

（8）在TaskRunner执行任务完成时，向DriverEndpoint终端点发送状态变更StatusUpdate消息，当DriverEndpoint终端点接收到该消息时，调用TaskSchedulerImpl的statusUpdate方法，根据任务执行不同的结果进行处理，处理完后再向该Executor分配执行任务。

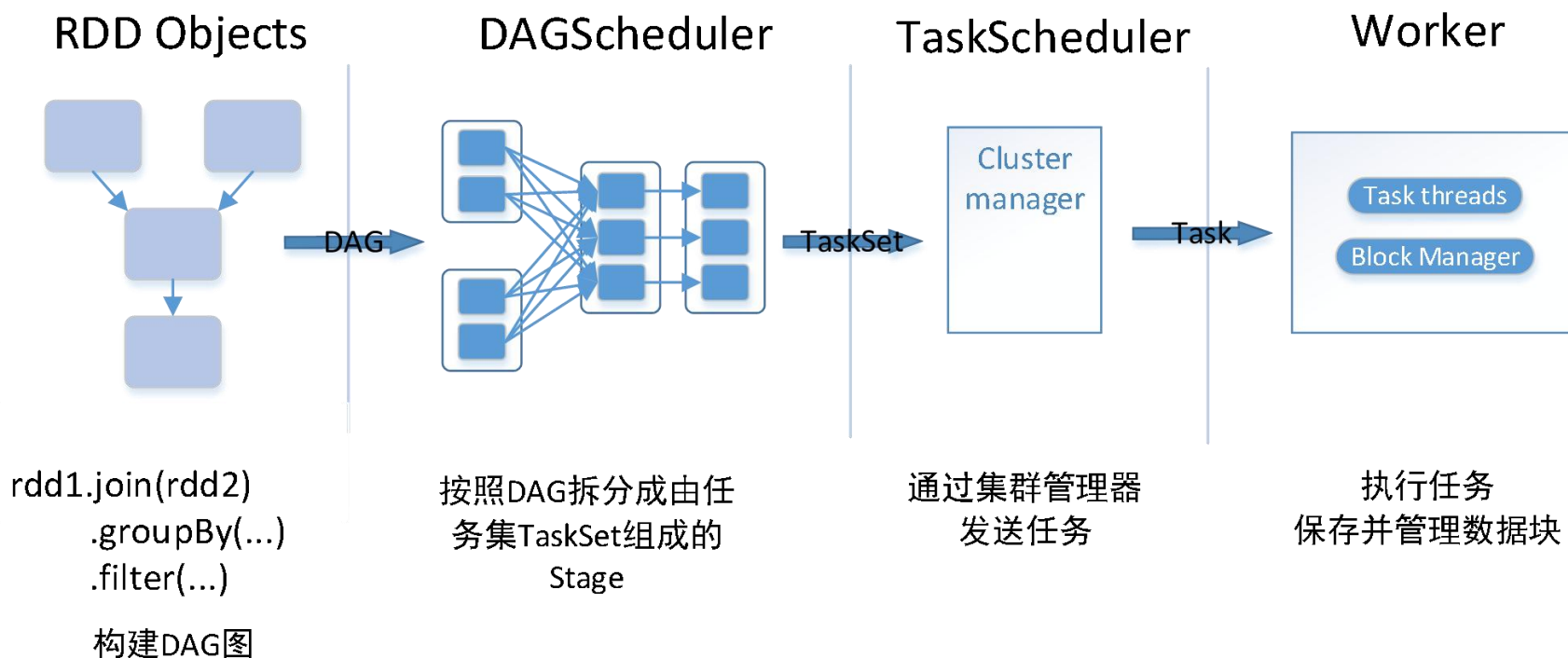


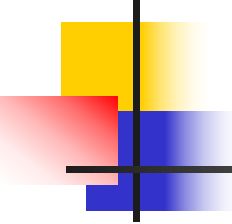
1.3.3 执行步骤

Spark的作业在其内部的调度主要是指基于RDD的一系列操作构成的作业在执行器中执行，主要包括四个步骤。

1.3.3 执行步骤

Spark内部调度的流程





1.3.3 执行步骤

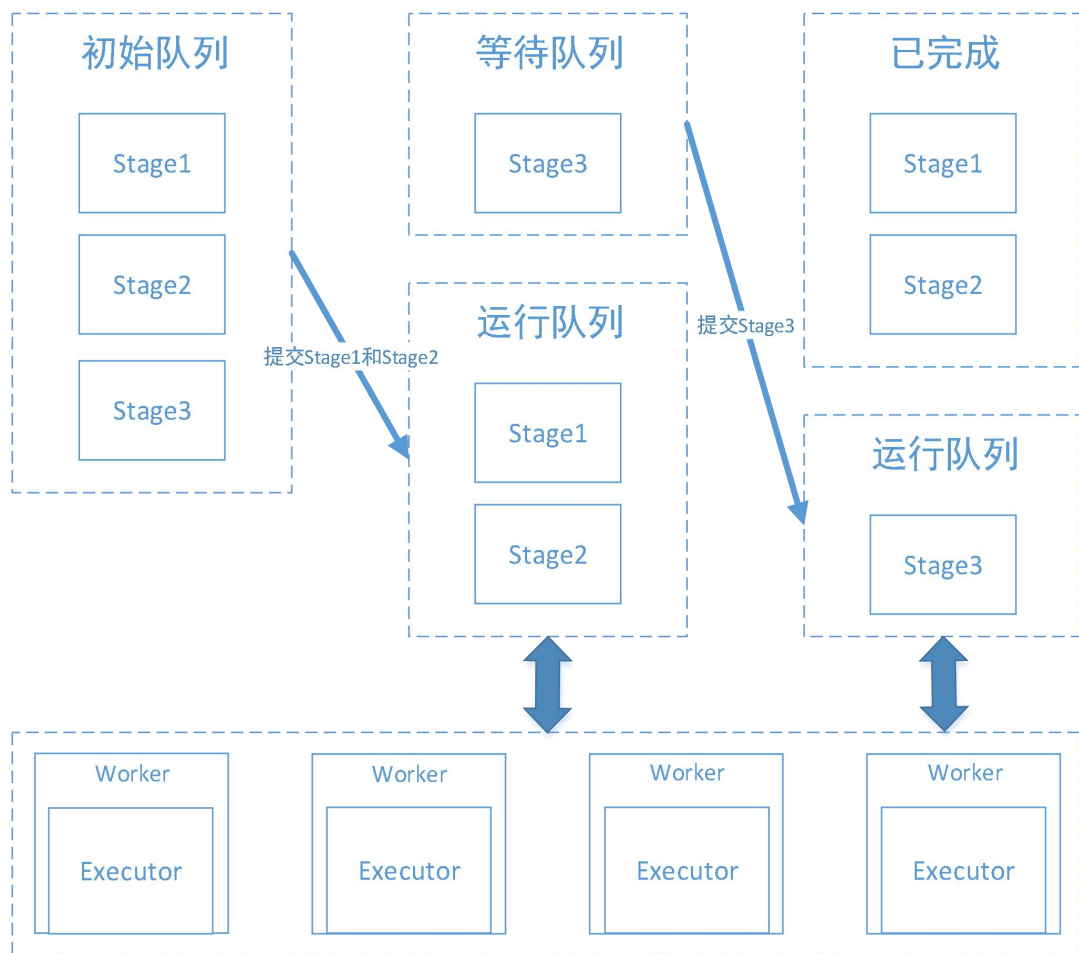
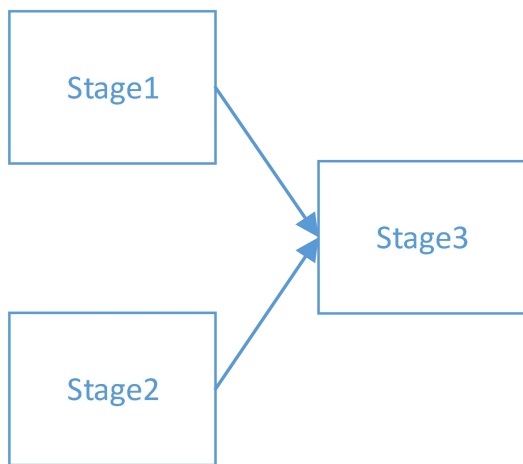
Spark内部调度的流程

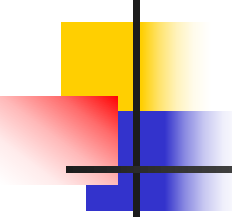
(1) 构建DAG图。Spark应用程序进行很多RDD操作，其中包括各种转换操作，它们描述了RDD之间的依赖关系，当遇到行动操作触发Job的提交，提交的是根据RDD依赖关系构建的DAG图，DAG图提交给DAGScheduler进行解析。

1.3.3 执行步骤

Spark内部调度的流程

- Stage依赖关系（左）
- Stage提交顺序（右）

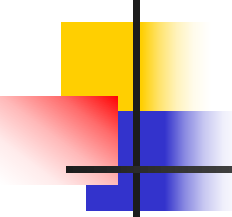




1.3.3 执行步骤

Spark内部调度的流程

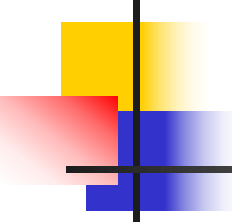
(2) DAGScheduler进行任务划分。DAGScheduler会收到之前生成的DAG图，然后对该图进行拆分，拆分的依据是DAG图中一个RDD到下一个RDD之间的操作步骤是否为宽依赖，是宽依赖则拆开为Stage。Stage的提交是有顺序的，再Stage依赖关系中，它的Stage提交逻辑的Stage1与Stage2是可以并行的，在有计算资源的情况下会首先被提交，并且在这两个Stage都计算完成的情况下，再提交Stage3。



1.3.3 执行步骤

Spark内部调度的流程

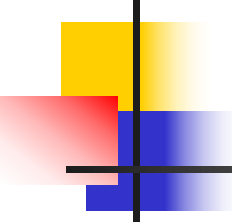
(3) TaskScheduler调度任务。TaskScheduler接收了来自DAGScheduler发送的TaskSet，然后把收到的TaskSet中的Task（Task的数量与输入的RDD的partition数量一样，partition是RDD的最小单元，RDD是分布在各个节点上的partition组成，因为每一个task只是处理一个partition上的数据），发送到集群Worker节点上去执行。



1.3.3 执行步骤

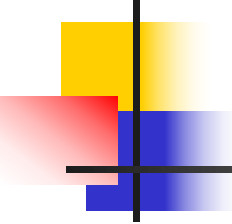
Spark内部调度的流程

(4) Worker执行任务。Worker节点中的Executor收到TaskScheduler发送过来的Task后会执行Task。Task以多线程方式运行，每个线程负责一个任务。



1.3.4 Spark运行模式

Spark的运行模式多种多样，灵活多变，部署在单机上时，可以用本地模式运行；而当以分布式集群的方式部署时，也有众多的运行模式可供选择，这取决于集群的实际情况，底层的资源调度即可以依赖外部资源调度框架，也可以使用Spark内部的Standalone模式。对于外部资源调度框架，目前支持相对稳定的Hadoop YARN模式和Mesos模式。



1.3.4 Spark运行模式

本地（Local）模式

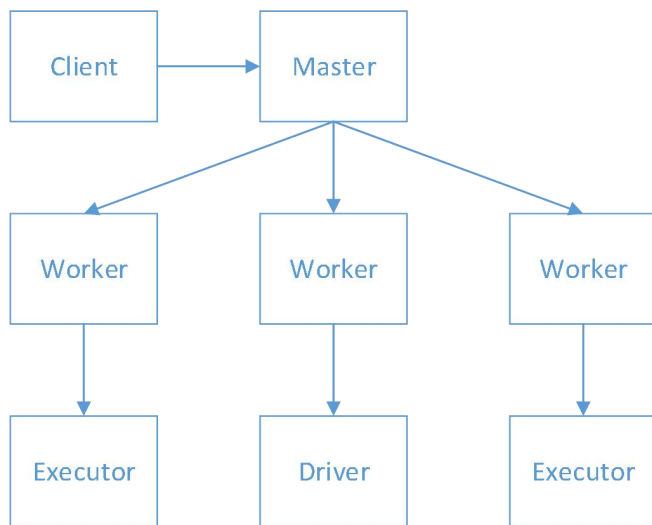
在本地运行模式中，**Spark**的所有进程都在一台机器上的**JVM**上运行。在本地运行模式下，作业划分调度后，任务集会发送到本地终端点，本地终端接收到任务后，会在本地启动**Executor**，这一切工作都在本地执行。

1.3.4 Spark运行模式

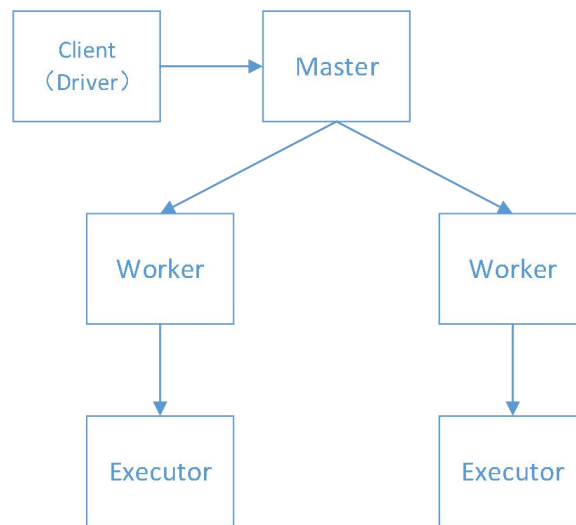
独立（Standalone）集群运行模式

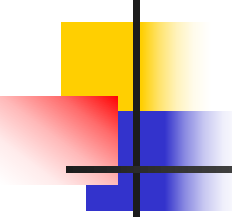
■ Standalone的Spark结构图

Driver run on Worker



Driver run on client





1.3.4 Spark运行模式

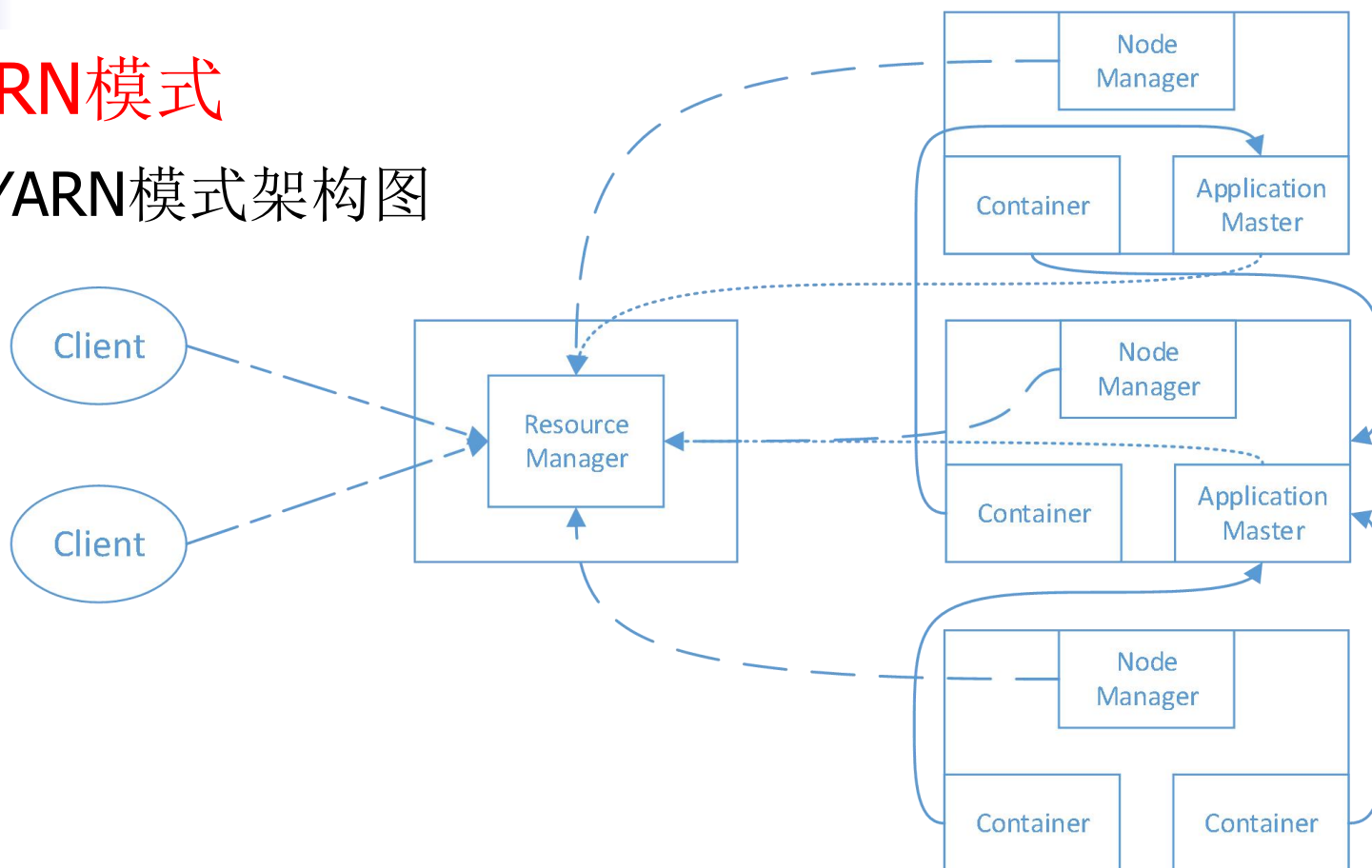
独立（Standalone）集群运行模式

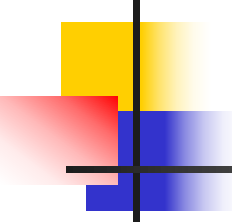
Standalone模式即独立模式，是Spark自带的一种简单集群管理器，可单独部署到一个集群中，无需依赖其他任何资源管理系统。从一定程度上说，它是Spark on YARN 和Spark on Mesos 的基础。它采用Master/Slave的典型架构，其组成有Client（本地客户端）节点、Master节点、Worker节点。Driver不仅可以在Worker节点运行，也可以在Client上运行

1.3.4 Spark运行模式

YARN模式

■ YARN模式架构图

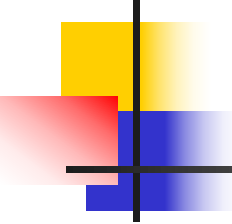




1.3.4 Spark运行模式

YARN模式

Spark若与别的分布式应用共享集群，Spark就需要运行在集群管理器上（例如Spark框架与MapReduce框架同时运行，如果不用第三方资源管理器进行资源分配，MapReduce分到的内存资源会很少，效率低下）。下面将介绍Spark运行在第三方资源管理器上的部署方案。

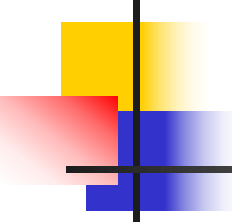


1.3.4 Spark运行模式

YARN模式

Spark若与别的分布式应用共享集群，Spark就需要运行在集群管理器上（例如Spark框架与MapReduce框架同时运行，如果不用第三方资源管理器进行资源分配，MapReduce分到的内存资源会很少，效率低下）。

资源管理器（Yet Another Resource Negotiator，YARN）是一个通用的资源管理系统，能够为上层应用提供统一资源管理和资源调度。YARN为集群在利用率、资源统一管理和数据共享等方面带来了巨大的好处。

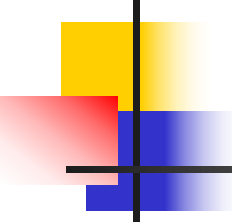


1.3.4 Spark运行模式

YARN模式

YARN 将资源管理和作业调度及监控分成了两个独立的服务程序：全局的资源管理（Resource Manager，RM）和针对个人应用的Master（Application Master，AM）

。

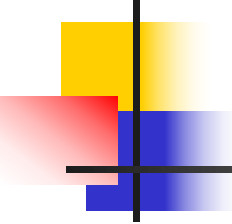


1.3.4 Spark运行模式

YARN模式架构

ResourceManager (RM)：负责全局资源管理，接收Client端任务请求，接收和监控NodeManager的资源情况汇报，负责资源的分配与调度，启动和监控ApplicationMaster。

NodeManager (NM)：可以看作节点的资源和管理器，启动Container运行Task计算，汇报资源、Container情况给RM，汇报任务处理情况给AM。



1.3.4 Spark运行模式

YARN模式架构

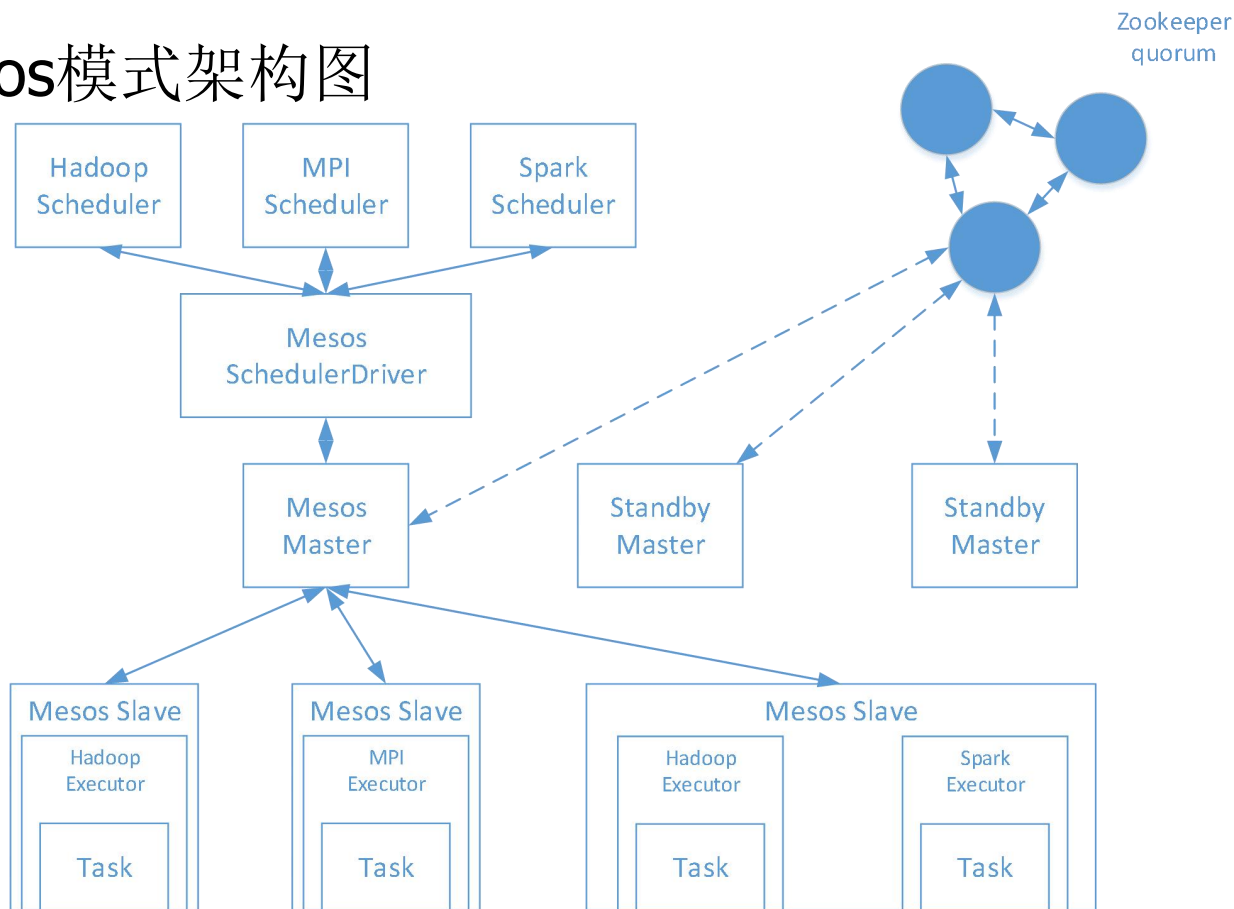
ApplicationMaster (AM)：主要是负责单个Application (Job) 的Task管理和调度，向RM申请资源，向NM发出launch Container指令，接收NM的Task处理状态信息。

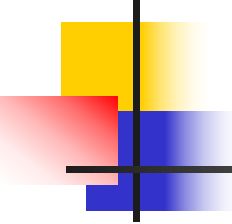
Container：YARN中资源分配单位，资源使用Container表示，每个任务占用一个Container，并在Container中运行。

1.3.4 Spark运行模式

Mesos模式

■ Mesos模式架构图

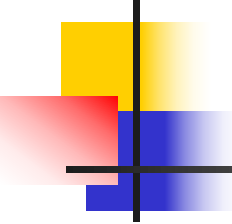




1.3.4 Spark运行模式

Mesos模式

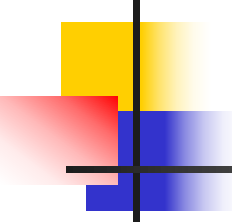
Mesos是Apache旗下的开源软件，采用Master/Slave结构，作为Apache下的开源分布式资源管理框架，被称为分布式系统的内核。Mesos是一个集群管理器，提供有效的跨分布式应用或框架的资源隔离共享，可以支持Hadoop、MPI、Hypertable、Spark等。由于Mesos和Spark都是Apache旗下的开源项目，Spark运行在Mesos上相比运行在YARN上更加灵活。



1.3.4 Spark运行模式

Apache Mesos组件: Mesos Master

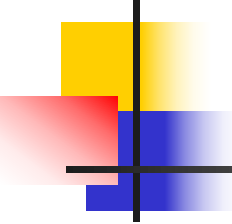
Mesos Master是整个系统的核心，负责管理接入到Mesos的各个计算框架的Slave，并将Slave上的资源按照指定算法分配给计算框架。由于Master存在单点故障的问题，因此Mesos采用了ZooKeeper解决该问题。



1.3.4 Spark运行模式

Apache Mesos组件：Mesos Slave

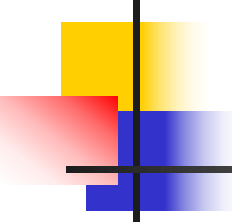
Mesos Slave负责接收并执行来自Mesos Master的命令，同时也管理节点上的任务，为各个Task分配资源。Slave在启动的时候将自己的资源量发送给Master。当计算框架注册时由Master的Allocator模块决定将哪些资源分配给该框架。当前考虑的资源有CPU和内存两种，也就是说，Slave会将CPU个数和内存量发送给Master，而用户提交作业时，需要指定每个任务需要的CPU个数和内存量，当任务运行时，Master会将任务放到包含固定资源的运行容器中运行，以达到资源隔离的效果。



1.3.4 Spark运行模式

Apache Mesos组件：框架

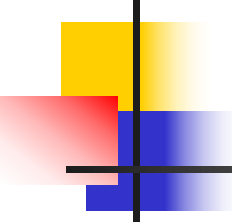
整个Mesos系统采用了双层调度框架：第一层，由Mesos将资源分配给各个计算框架；第二层，各个计算框架的调度器将资源分配给其内部的任务。Mesos为了向各种调度器提供统一的接入方式，在Mesos内部实现了一个调度器驱动器（MesosSchedulerDriver），计算框架的调度器可调用该驱动器中的接口与Master进行交互，完成一系列功能（如注册、资源分配等）。



1.3.4 Spark运行模式

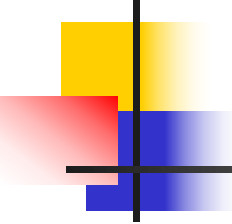
Apache Mesos组件：执行容器

执行容器主要用于启动计算框架内部的任务。由于不同的框架，启动任务的接口或者方式不同，当一个新的框架要接入Mesos时，需要重写该执行容器，告诉Mesos如何启动该框架中的任务。为了向各种框架提供统一的执行器编写方式，Mesos内部实现了一个执行器驱动器（MesosExecutorDriver），计算框架可通过该驱动器的相关接口告诉Mesos启动的方法和任务。



1.4 WordCount示例

Spark支持多种开发语言如Scala、Java、Python和R编写程序，而且还支持在Scala、Python、R的shell中进行交互式查询。接下来通过不同语言编写单词统计（WordCount）为例，展示不同语言的Spark编程。



1.4.1 三种编程语言的示例程序

WordCount以单词统计应用为例，将文件内所有单词以空格为切分，然后统计每个单词的出现次数，保存为文本形式。假设输入文件内容为：

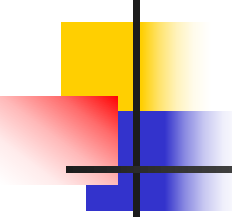
a b

d e

c c

a c

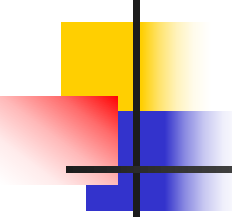
e e



1.4.1 三种编程语言的示例程序

Scala版本

```
import org.apache.spark.{SparkConf,SparkContext}
//建立的 WordCount 对象，以及定义 main 函数
object WordCount {
    def main(args: Array[String]): Unit = {
        val conf = new SparkConf().setMaster("local").setAppName("wordcount")
        val sc = new SparkContext(conf)
        val lines = sc.textFile("文件路径")
        val words = lines.flatMap(line => line.split(" "))
        val count = words.map(word => (word,1)).reduceByKey{case(x,y) => x+y}
        println(count.collect().mkString("\n"))    //逐行输出
    }
}
```

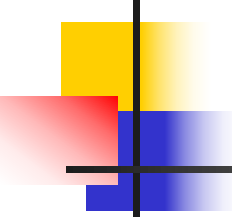


1.4.1 三种编程语言的示例程序

Python版本

在命令行中输入**pyspark**（在**Spark**环境配置好时直接输入**pyspark**），然后输入下面代码。

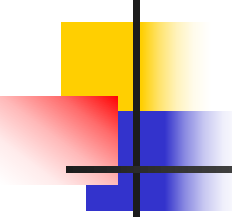
```
from pyspark import SparkConf, SparkContext as sc
conf = SparkConf().setMaster("local").setAppName("wordcount")
sc=SparkContext.getOrCreate(conf)
lines = sc.textFile("文件路径")
words = lines.flatMap(lambda line: line.split(" "))
count = words.map(lambda x:(x,1)).reduceByKey(lambda x,y: x+y)
print(count.collect())
```



1.4.1 三种编程语言的示例程序

Java版本

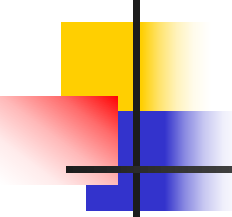
```
import java.util.Arrays;
import java.util.Iterator;
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.FlatMapFunction;
import org.apache.spark.api.java.function.Function2;
import org.apache.spark.api.java.function.PairFunction;
import org.apache.spark.api.java.function.VoidFunction;
import scala.Tuple2;
```

1.4.1 三种编程语言的示例程序

Java版本

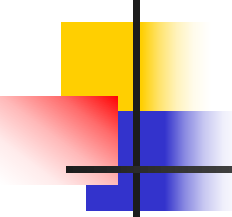
```
public class javaWordCount {  
    public static void main(String[] args) {  
        // 第一步：初始化配置  
        SparkConf conf = new SparkConf().setMaster("local").setAppName("wordcount");  
        // 第二步：创建 JavaSparkContext 对象，SparkContext 是 Spark 的所有功能的入口  
        JavaSparkContext sc = new JavaSparkContext(conf);  
  
        // 第三步：创建一个初始的 RDD  
        // SparkContext 中用于根据文件类型的输入源创建 RDD 的方法，叫做 textFile()方法  
        JavaRDD<String> lines = sc.textFile("文件路径"); //括号中为文件路径  
  
        // 第四步：对初始的 RDD 进行 transformation 操作，也就是词频统计初始操作  
        // 首先把单词用空格拆开|  
        JavaRDD<String> words = lines.flatMap(new FlatMapFunction<String, String>() {  
            private static final long serialVersionUID = 1L;  
            @Override  
            public Iterator<String> call(String line) throws Exception {  
                return Arrays.asList(line.split(" ")).iterator();  
            }  
        });  
    }  
};
```



1.4.1 三种编程语言的示例程序

Java版本

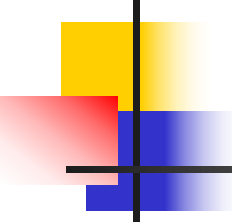
```
// 将每一个单词，映射为（单词，1）的这种格式
JavaPairRDD<String, Integer> pairs = words.mapToPair(new PairFunction<String,
String, Integer>() {
    private static final long serialVersionUID = 1L;
    @Override
    public Tuple2<String, Integer> call(String word) throws Exception {
        return new Tuple2<String, Integer>(word, 1);
    }
});
// 以单词作为 key，统计每个单词出现的次数
JavaPairRDD<String, Integer> wordCounts = pairs.reduceByKey(new
Function2<Integer, Integer, Integer>() {
    private static final long serialVersionUID = 1L;
    @Override
    public Integer call(Integer v1, Integer v2) throws Exception {
        return v1 + v2;
    }
});
```



1.4.1 三种编程语言的示例程序

Java版本

```
// 输出 WordCount 结果
wordCounts.foreach(new VoidFunction<Tuple2<String,Integer>>() {
    private static final long serialVersionUID = 1L;
    @Override
    public void call(Tuple2<String, Integer> wordCount) throws Exception {
        System.out.println(wordCount);
    }
});
sc.close();
}
```



1.4.1 三种编程语言的示例程序

WordCount输出

```
(d,1)
```

```
(e,3)
```

```
(a,2)
```

```
(b,1)
```

```
(c,3)
```

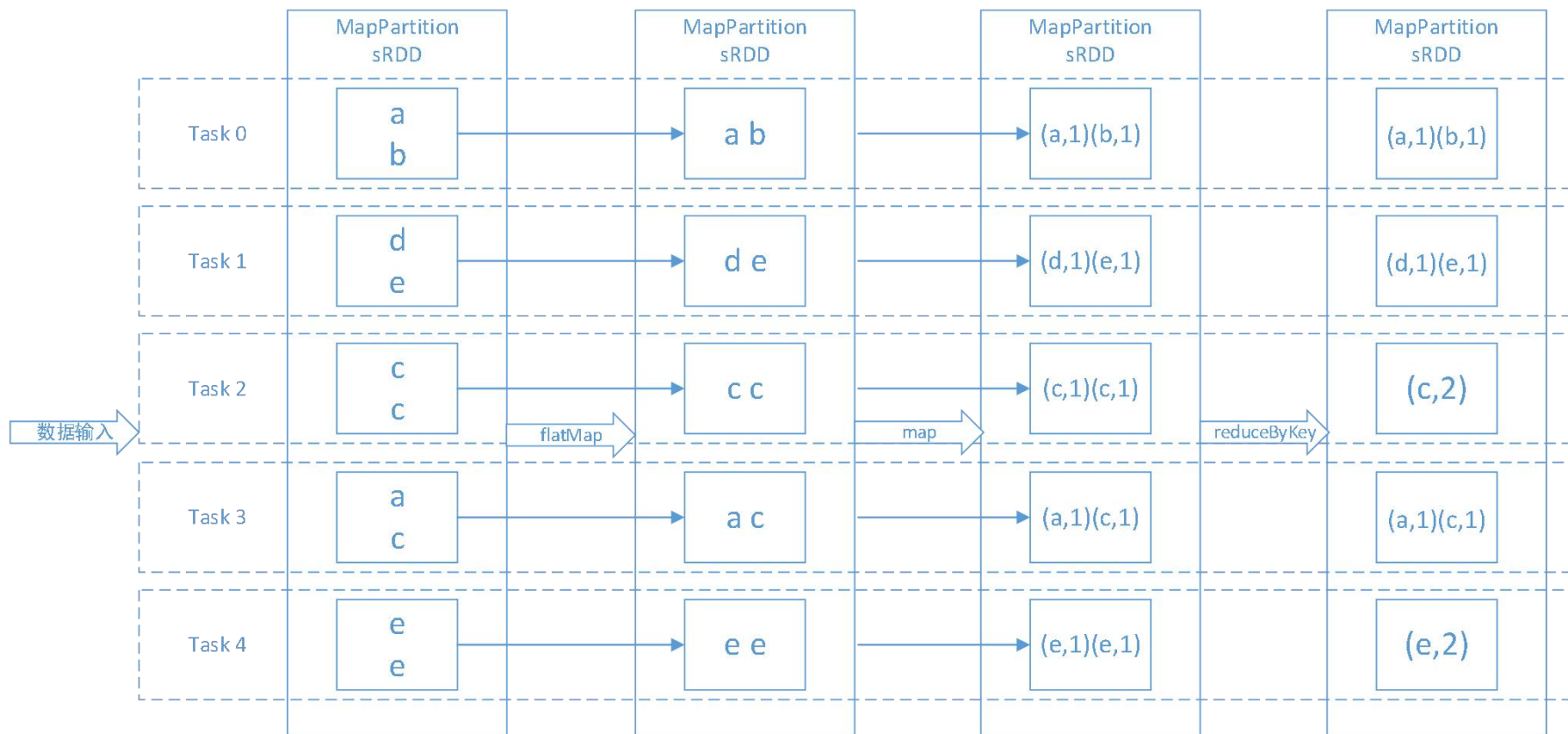
1.4.2 Scala版本WordCount运行分析

WordCount示例代码

```
val lines = sc.textFile("文件路径") //获取文件
val words = lines.flatMap(line => line.split(" ")) //用空格切分 RDD 成一个个单词 RDD
val pairs = words.map(word => (word,1)) //把单词化成(word,1)的形式
val WordCounts = pairs.reduceByKey(_+_ ) //进行单词统计，统计相同单词数量
```

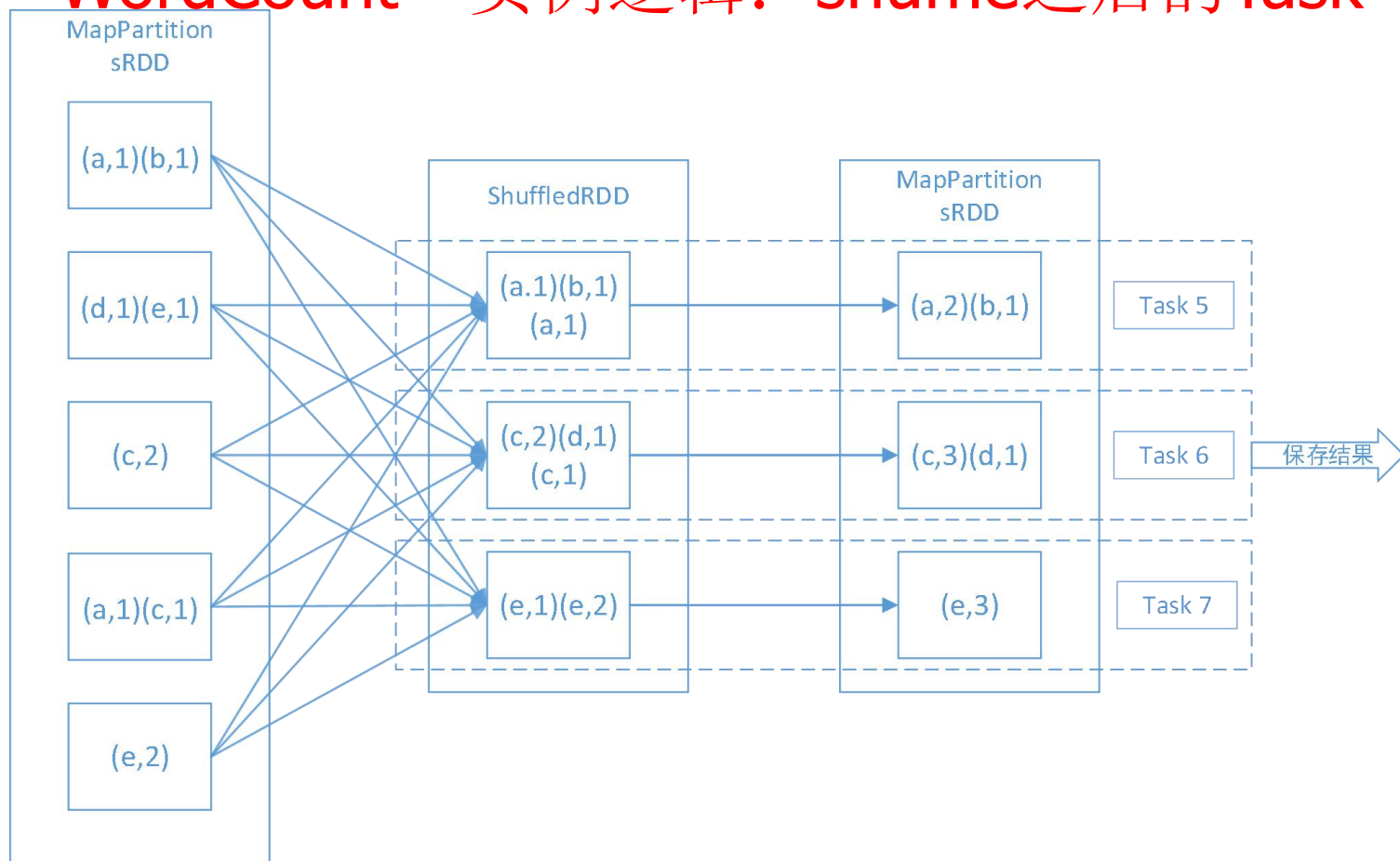
1.4.2 Scala版本WordCount运行分析

“WordCount” 实例逻辑：shuffle之前的Task



1.4.2 Scala版本WordCount运行分析

“WordCount” 实例逻辑：shuffle之后的Task

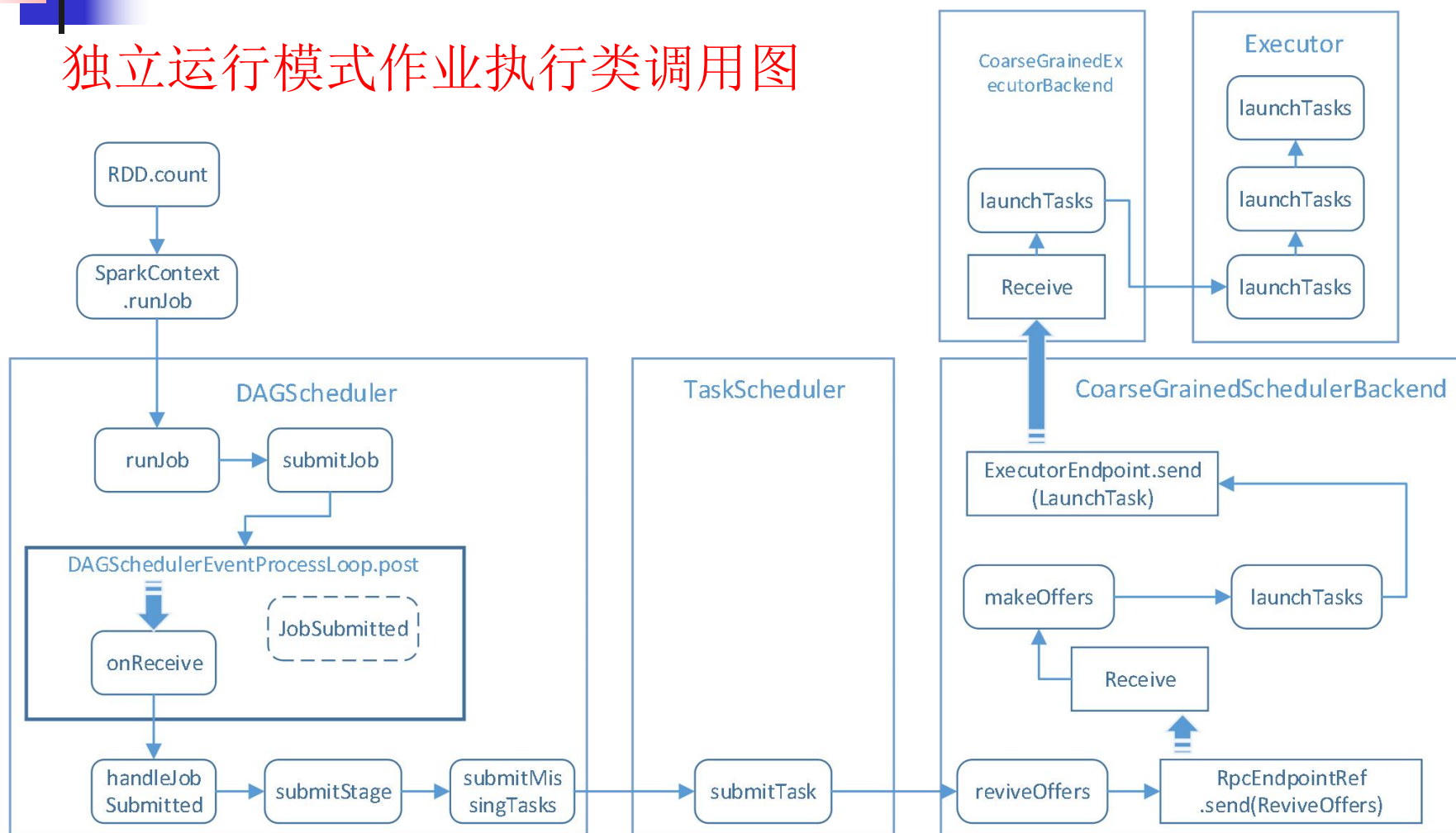


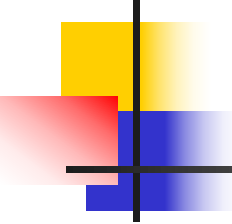
1.4.2 Scala版本WordCount运行分析

“WordCount”根据依赖划分为两类Task，一类是Shuffle操作之前的Task，一类是Shuffle操作之后输出结果的Task，在Shuffle操作之前，初始的RDD有5个Partition，因此有5个Task生成，Shuffle操作输出的Partition数量是3个，所以Shuffle操作之后有3个Task，它的基本操作步骤和上述的代码注释解释的一样，在转换操作reduceByKey时会触发Shuffle过程，然而在这个过程开始之前，有一个本地聚合的过程，比如在Task 2中的Partition的(c,1)(c,1)聚合成了(c,2)。Shuffle的结果为下游的三个Partition，之后再做一个聚合，就生成了结果RDD。

1.4.3 WordCount中类调用关系

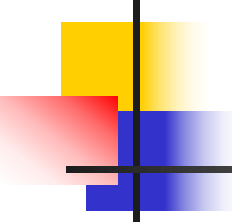
独立运行模式作业执行类调用图





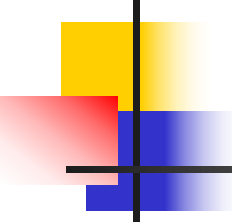
1.4.3 WordCount中类调用关系

在经典的WordCount的例子中，作业真正提交是从“count”这个行动操作开始的，在RDD源代码的count方法中触发了SparkCount的runJob方法提交作业，对于RDD来说，它根据彼此之间的依赖形成一个DAG，然后把这个DAG交给DAGScheduler处理，DAGScheduler类内部首先在runJob方法里调用submitJob方法继续提交作业，这里会发生阻塞，直到返回作业完成或者失败的结果；然后在submitJob方法中创建一个JobWaiter对象，借助内部消息处理把对象发送给DAGScheduler内嵌类DAGSchedulerEventProcessLoop进行处理；最后在它的消息接收方法OnReceive中，接收到JobSubmitted样例完成匹配后，继续调用DAGScheduler的handleJobSubmitted方法来提交作业，阶段划分就是在该方法中进行的。



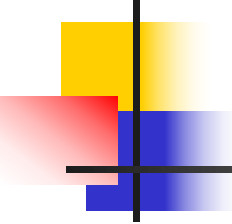
1.4.3 WordCount中类调用关系

在提交调度阶段开始时，会依据调度阶段找父调度阶段，直到找到不存在父调度阶段，则使用**submitMissingTasks**方法提交，存在则放入等待列表中。当调度阶段提交运行后，在**DAGScheduler**的**submitMissingTasks**方法中根据**Partition**个数拆分对应个数的任务，任务会提交到**TaskScheduler**处理，对于每一个任务集，都包含了对应调度阶段的所有任务，这些任务处理逻辑上完全一样，只是处理的数据不同。



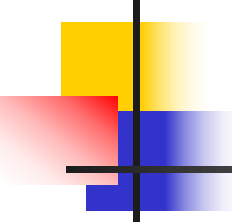
1.4.3 WordCount中类调用关系

在Spark独立运行模式作业执行类调用图中，当TaskScheduler收到发送过来的任务集时，在submitTasks方法中构建一个TaskSetManager实例，用来管理这个任务集的生命周期，接着在CoarseGrainedSchedulerBackend类方法中的reviveOffers方法，向DriverEndPoint终端点发送消息，调用makeOffers方法。该方法会获取集群中可用的Executor，然后向任务集的任务分配运行资源，最后提交到launchTasks方法中，该方法把任务全部发送到Worker节点上CoarseGrainedSchedulerBackend，CoarseGrainedSchedulerBackend接收到LaunchTask消息时，会调用Executor的launchTask方法进行处理，在Executor的launchTask方法中会初始化一个TaskRunner来封装任务，用于管理任务运行时的细节，TaskRunner的run方法中，首先会对发送过来的Task本身以及它所依赖的Jar等文件进行反序列，最后反序列化的任务调用Task的runTask方法。



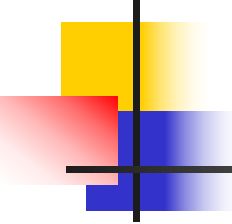
1.5 本章小结

Spark已经成为广泛采用的大数据处理平台中的佼佼者，本章初步介绍了**Spark**的发展历程以及它的重要组成部分。然后介绍了**Spark**的运行原理；接着介绍了**Spark**所支持的运行模式，包括**Spark**自带的集群管理器，或在YARN或Mesos这种第三方集群管理器上运行**Spark**；最后结合WordCount程序为例进行了**Spark**的编程和工作机制的介绍。



1.6 本章习题

1. Spark有哪些优点？
2. Spark包含哪些组件？
3. 为什么要采用第三方资源管理器管理集群？
4. Job、Stage、Task之间有什么关系？
5. Standalone模式中Driver可以在哪里运行？
6. DAGScheduler在Spark运行过程中有什么作用？



1.6 本章习题

7. Mesos模式中Mesos Slave有什么作用？
8. 概述Spark的执行流程。
9. 介绍ResourceManager（RM）和NodeManager（NM）在YARN模式中的作用。
10. 解释WordCount的运行过程。



Spark大数据 编程基础 (Scala版)

